Semana 7 · TLS 1.3 híbrido sobre I2C

Fase de la propuesta: intercambio híbrido y pruebas de confidencialidad (semanas 7-8).

Objetivo	Establecer sesiones TLS 1.3 con el intercambio **híbrido X25519MLKEM768** sobre I2C, verificar que realmente se negocia el grupo híbrido y que el ESP32 tiene memoria suficiente.
Dedicación estimada	4–6 horas.
Antes de empezar	Semana 6 cerrada (TLS clásico estable).
Al terminar tendrán	Corridas híbridas estables, primera comparación clásico vs híbrido y la etiqueta v0.7 en Git.

Paso 1. Primera sesión híbrida

```
cd ~/proyecto-kyber-rpi/pi && source ../src/venv/bin/activate
python3 servidor_tls.py --grupo hibrido --frame F
```

Campo	Esperado con el híbrido	Por qué
grupo_detectado	hibrido	el ServerHello lleva X25519 (32 B) + texto cifrado ML-KEM-768 (1.088 B)
bytes_clienthello	~1.380 B (clásico: ~200 B)	la clave pública ML-KEM-768 ocupa 1.184 B
bytes_vuelo_srv	~1.800 B (clásico: ~750 B)	incluye el texto cifrado ML-KEM
esp_estado / error	0 / vacío	handshake y sesión completos

> **Si aparece BAD_ECPOINT o esp_estado distinto de 0 solo en el híbrido**
>
> Confirmen en la consola serie que el ESP32 dice **wolfSSL 5.9.4**. Con versiones 5.8.x hay ~3 % de rechazos de interoperabilidad con OpenSSL 3.5 (verificado). Si la versión es correcta y el error sigue, pásenme el JSON completo.

Paso 2. Memoria del ESP32

ML-KEM necesita más memoria que X25519. Revisen `esp_stack_libre` (pila libre mínima de la tarea TLS) y `esp_heap_usado_hs` (memoria de la sesión).

- Si `esp_stack_libre` es menor a ~2.000 bytes, o la consola muestra `stack overflow` / `Guru Meditation Error`: en `main.c` suban la pila de la tarea TLS de `32768` a `49152` (línea `xTaskCreatePinnedToCore(tarea_tls, "tls", ...)`), regraben y repitan.
- Anoten ambos valores: la diferencia de memoria entre clásico e híbrido es un resultado de la tesis.

Paso 3. Estabilidad mezclando ambos grupos

```
cd ~/proyecto-kyber-rpi/pi && source ../src/venv/bin/activate
python3 campana.py --bloque prueba_hibrido --grupos clasico,hibrido --tamanos 128 \
                   --reps 15 --reposo 10 --pausa 1 --frame F
python3 resumen_rapido.py
```

Esperado: 30 corridas correctas y `Consistencia grupo pedido vs detectado: OK`.

Paso 4. Primera comparación (no es el resultado final)

Con la salida de `resumen_rapido.py` llenen esta tabla en la bitácora. Todavía es una muestra pequeña; sirve para revisar que los números tengan sentido antes de la campaña.

Medida (media)	Clásico	Híbrido	Diferencia (%)
esp_t_hs_us (duración del handshake)			
bytes_clienthello			
esp_heap_usado_hs			
esp_stack_libre			
rend_util_kbps			

Paso 5. Marcar el hito en Git

```
cd ~/proyecto-kyber-rpi
git add -A && git commit -m "Semana 7: TLS híbrido estable sobre I2C"
git tag -a v0.7-hibrido -m "Handshake híbrido estable sobre I2C"
git push && git push origin v0.7-hibrido
```

Criterio de cierre de la semana

- ☐ Sesión híbrida completa con grupo_detectado = hibrido.
- ☐ ClientHello híbrido ~1.380 B y clásico ~200 B confirmados en el hardware.
- ☐ Pila del ESP32 con margen (> ~2 KB libres) y sin reinicios.
- ☐ 30 corridas mezcladas correctas y consistentes.
- ☐ Tabla de primera comparación en la bitácora; etiqueta v0.7-hibrido publicada.